

<div align="center">

Job Referral Network

A production-ready graph-powered job referral platform built with Node.js, Express, React, and CognoDB.

NODE.JS	18+	https://nodejs.org
REACT	18	https://reactjs.org
COGNODB	GRAPH DB	https://cognodb.com
EXPRESS.JS	5	https://expressjs.com

Discover job opportunities through the power of your professional network using multi-hop graph traversal — something impossible to express cleanly in a relational database.

</div>

Table of Contents

- [Project Overview](#)
- [Why Graph Database?](#)
- [Graph Data Model](#)
- [Architecture](#)
- [Folder Structure](#)
- [Prerequisites](#)
- [Environment Variables](#)
- [Installation & Setup](#)
- [Running Locally](#)
- [Seeding the Database](#)
- [API Documentation](#)
- [Cypher Query Examples](#)
- [Frontend Pages](#)
- [Deployment](#)

Project Overview

The **Job Referral Network** is a full-stack web application that leverages a graph database to help professionals discover job opportunities through their connections. Instead of simple keyword matching, the platform traverses the professional graph to answer questions like:

- "Which companies are my connections working at that are actively hiring?"
- "Which jobs match my exact skills, and do I know anyone inside?"
- "Who in my extended network (2nd or 3rd degree) shares my skill set?"

These queries require traversing multiple relationship hops across interconnected entities — the ideal use case for a graph database.

Why Graph Database?

Relational databases store data in flat tables connected by foreign keys. Answering the question "Find jobs posted by companies where my friends' friends work, that require my skills" in SQL requires:

```
SELECT j.* FROM jobs j
JOIN companies c ON j.company_id = c.id
JOIN users u2 ON u2.company_id = c.id
JOIN friendships f1 ON f1.friend_id = u2.id
JOIN friendships f2 ON f2.friend_id = f1.user_id
JOIN user_skills us ON us.user_id = f2.user_id
JOIN skills s ON s.id = us.skill_id
JOIN job_skills js ON js.skill_id = s.id AND js.job_id = j.id
WHERE f2.user_id = $currentUser
```

This becomes **exponentially slower** as data grows. In CognoDB (Cypher), the same query is:

```
MATCH (u:User {id: $userId})-[:KNOWS*1..3]->(friend)-[:WORKS_AT]->(c:Company)-[:HIRING_FOR]->(j:Job)
WHERE (u)-[:HAS_SKILL]->(:Skill)-[:REQUIRES]->(j)
RETURN DISTINCT j, c
```

Key advantages of graph databases for this use case:

Feature	Relational DB	Graph DB
Multi-hop relationship queries	Slow, complex JOINS	Native, fast traversal
Adding new relationship types	Requires schema migration	Add relationship, done
Pattern matching	Requires subqueries	Declarative Cypher patterns
Friend-of-friend queries	O(n ³) or worse	Near O(1) with index
Recommendation engines	Very difficult	Natural fit

Graph Data Model

```
(User) -[:KNOWS]-----> (User)
|
|[:WORKS_AT]           |[:HAS_SKILL]
v                       v
(Company)              (Skill)
|                       ^
|[:HIRING_FOR]         |[:REQUIRES]
v                       |
(Job)-----/
```

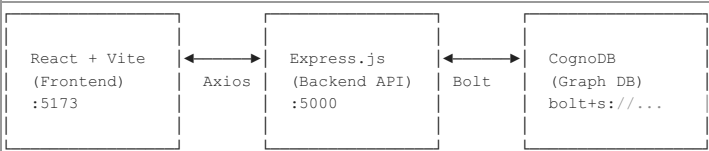
Nodes

Node	Properties
User	id, name, email, experience (years)
Company	id, name, location
Job	id, title, location, salary
Skill	id, name

Relationships

Relationship	From → To	Meaning
[:KNOWS]	User → User	Professional connection (bidirectional)
[:WORKS_AT]	User → Company	Employment
[:HAS_SKILL]	User → Skill	Skill proficiency
[:HIRING_FOR]	Company → Job	Open position
[:REQUIRES]	Job → Skill	Required skill for a job

🏗 Architecture



Backend Design Pattern — MVC:

```
Request → Route → Controller → Service (Cypher) → CognoDB
      |
      v
      Middleware (Error Handling)
```

Tech Stack:

Layer	Technology
Frontend	React 18, Vite, Tailwind CSS v4, React Router, Axios
Backend	Node.js, Express.js 5, neo4j-driver
Database	CognoDB (managed Neo4j-compatible graph database)
Seeding	@faker-js/faker

📁 Folder Structure

```
wexa/
├── backend/
│   ├── config/
│   │   └── db.js # Driver init, session factory, retry logic
│   ├── controllers/
│   │   ├── userController.js # CRUD handlers for User nodes
│   │   ├── companyController.js # CRUD handlers for Company nodes
│   │   ├── jobController.js # CRUD handlers for Job nodes
│   │   ├── skillController.js # CRUD handlers for Skill nodes
│   │   └── graphController.js # Graph traversal handlers
│   ├── middlewares/
│   │   └── errorHandler.js # Global error & 404 middleware
│   ├── routes/
│   │   ├── index.js # Root API router
│   │   ├── userRoutes.js
│   │   ├── companyRoutes.js
│   │   ├── jobRoutes.js
│   │   ├── skillRoutes.js
│   │   └── graphRoutes.js # Graph query API endpoints
│   ├── seed/
│   │   └── seed.js # Generates 20 users, 8 cos, 20 skills, 15 jobs
│   ├── services/
│   │   ├── userService.js # Parameterised Cypher for User CRUD
│   │   ├── companyService.js
│   │   ├── jobService.js
│   │   ├── skillService.js
│   │   └── graphService.js # All multi-hop graph traversal queries
│   ├── .env.example
│   ├── app.js # Express app, middleware, routes wiring
│   └── server.js # Entry point - DB verify then listen
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   └── UserSelect.jsx # Custom dark-themed searchable dropdown
│   │   ├── pages/
│   │   │   ├── Dashboard.jsx # Stats overview + graph model info
│   │   │   ├── NetworkExplorer.jsx # Browse 1st-degree connections
│   │   │   └── Recommendations.jsx # Multi-hop job recommendation engine
│   │   ├── services/
│   │   │   └── api.js # Axios base client
│   │   ├── App.jsx # Router + collapsible sidebar layout
│   │   ├── index.css # Global dark theme, glassmorphism styles
│   │   └── main.jsx
│   ├── .env
│   ├── postcss.config.js
│   ├── tailwind.config.js
│   └── vite.config.js
└── README.md
```

✓ Prerequisites

- **Node.js** v16 or higher (`node -v`)
- **npm** v8 or higher (`npm -v`)
- A free **CognoDB Cloud** instance → console.cognodb.com/signup (<https://console.cognodb.com/signup>)

🔑 Environment Variables

Backend — backend/.env

Copy from `.env.example` and fill in your CognoDB credentials:

```
# CognoDB Cloud connection (Bolt over TLS)
DB_URI=bolt+s://<your-instance-id>.databases.cognodb.cloud
DB_USERNAME=cognodb
DB_PASSWORD=your_password_here

# Express server port
PORT=5000
```

Frontend — frontend/.env

```
# Backend API base URL
VITE_API_URL=http://localhost:5000/api
```

For production: set `VITE_API_URL` to your deployed Render backend URL.

📦 Installation & Setup

1. Clone the repository

```
git clone <your-repo-url>
cd wexa
```

2. Install backend dependencies

```
cd backend
npm install
```

3. Install frontend dependencies

```
cd ../frontend
npm install
```

► Running Locally

Open **two terminals**:

Terminal 1 — Backend:

```
cd backend
npm run start      # production-like
# or
npm run dev        # with nodemon auto-reload
```

Terminal 2 — Frontend:

```
cd frontend
npm run dev
```

- Backend API: `http://localhost:5000`
- Frontend App: `http://localhost:5173`
- Health check `http://localhost:5000/health`

🔑 Seeding the Database

The `seed` script generates realistic mock data using `Faker` and inserts it into your `CognoDB` instance. Run it once after setting up your `.env`:

```
cd backend
node seed/seed.js
```

What gets created:

Entity	Count
Users	20
Companies	8
Skills	20
Jobs	15
KNOWS relationships	~60–100
WORKS_AT relationships	~16
HAS_SKILL relationships	~80–140
HIRING_FOR relationships	15
REQUIRES relationships	~45–75

⚠ The `seed` script runs `MATCH (n) DETACH DELETE n` first to clear existing data. Run it only once, or intentionally to reset.

📖 API Documentation

Base URL: `http://localhost:5000/api`

Users

Method	Endpoint	Description
GET	<code>/users</code>	List all users
GET	<code>/users/:id</code>	Get a single user by ID
POST	<code>/users</code>	Create a new user
PUT	<code>/users/:id</code>	Update a user
DELETE	<code>/users/:id</code>	Delete a user and all their relationships

POST `/users` body:

```
{ "name": "Jane Doe", "email": "jane@example.com", "experience": 5 }
```

Companies

Method	Endpoint	Description
GET	<code>/companies</code>	List all companies
POST	<code>/companies</code>	Create a new company

POST `/companies` body:

```
{ "name": "Acme Corp", "location": "San Francisco" }
```

Jobs

Method	Endpoint	Description
GET	/jobs	List all jobs
POST	/jobs	Create a new job

POST /jobs body:

```
{ "title": "Senior Engineer", "location": "Remote", "salary": 150000 }
```

Skills

Method	Endpoint	Description
GET	/skills	List all skills
POST	/skills	Create a new skill

POST /skills body:

```
{ "name": "GraphQL" }
```

Graph Queries

Method	Endpoint	Description
GET	/graph/connections/:userId	1st-degree connections (direct KNOWS)
GET	/graph/network/:userId	2nd and 3rd-degree connections
GET	/graph/referrals/:userId	Companies where connections work
GET	/graph/recommendations/:userId	Job recommendations (multi-hop)
GET	/graph/mutual/:user1/:user2	Mutual connections between two users
GET	/graph/skills/:userId	Users sharing similar skills

 Cypher Query Examples

1. First-degree Connections

```
MATCH (u:User {id: $userId})-[:KNOWS]->(connection:User)
RETURN connection
```

2. Extended Network (2nd & 3rd degree)

```
MATCH (u:User {id: $userId})-[:KNOWS*2..3]-(connection:User)
WHERE u <> connection AND NOT (u)-[:KNOWS]-(connection)
RETURN DISTINCT connection
```

3. Referral Opportunities

```
MATCH (u:User {id: $userId})-[:KNOWS]-(connection:User)-[:WORKS_AT]->(c:Company)
RETURN DISTINCT c AS company, collect(connection) AS contacts
```

4. Job Recommendations (Flagship Multi-hop Query)

```
-- Step 1: Find jobs that require skills the user has, aggregate skills first
MATCH (u:User {id: $userId})-[:HAS_SKILL]->(s:Skill)<-[:REQUIRES]-(j:Job)<-[:HIRING_FOR]-(c:Company)
WITH u, j, c, collect(DISTINCT s) AS matchedSkills

-- Step 2: Find insider contacts at the hiring company
OPTIONAL MATCH (u)-[:KNOWS]-(contact:User)-[:WORKS_AT]->(c)
RETURN j AS job, c AS company, matchedSkills, collect(DISTINCT contact) AS contacts
ORDER BY size(matchedSkills) DESC
```

The WITH clause between the two MATCH statements is critical — it prevents a cartesian product explosion where duplicate skills would be returned for each contact found.

5. Mutual Connections

```
MATCH (u1:User {id: $user1Id})-[:KNOWS]-(mutual:User)-[:KNOWS]-(u2:User {id: $user2Id})
RETURN DISTINCT mutual
```

6. Similar Skill Peers

```
MATCH (u:User {id: $userId})-[:HAS_SKILL]->(s:Skill)<-[:HAS_SKILL]-(other:User)
WHERE u <> other
RETURN other, collect(s) AS sharedSkills, size(collect(s)) AS score
ORDER BY score DESC
LIMIT 10
```

 Frontend Pages

Page	Route	Description
Dashboard	/	Live stats (users, companies, jobs, skills), graph model diagram, "Why Graph DB?"
Network Explorer	/network	Select a user and explore their 1st-degree professional connections with live search
Recommendations	/recommendations	Multi-hop graph recommendation engine with insider contact detection and skill matching

UI Features:

- 🌑 Dark glassmorphism design with animated sidebar
- 📊 Animated number counters on the dashboard
- 🦴 Skeleton loaders while data fetches
- 🔍 Live searchable user dropdown (custom component)
- 🔔 Toast notifications for success/error events
- 🔗 Insider contact highlighting on recommendation cards
- 📊 Smart match scoring (Strong / Good / Potential)
- 🔌 Auto-reconnect on database timeout (ECONNRESET handled silently)

🚀 Deployment

Backend → Render

1. Push your code to a GitHub repository.
2. Go to render.com (<https://render.com>) → **New Web Service**.
3. Connect your repository, set:
 - **Build Command:** `npm install`
 - **Start Command:** `npm run start`

4. Add environment variables in the Render dashboard:

```
DB_URI      = bolt+s://...
DB_USERNAME = cognodb
DB_PASSWORD = ...
PORT       = 5000
```

Frontend → Vercel

1. Go to vercel.com (<https://vercel.com>) → **New Project**.
2. Import your repository, set the **Root Directory** to `frontend`.
3. Add environment variable:

```
VITE_API_URL = https://your-backend.onrender.com/api
```

4. Deploy.

📸 Screenshots

Add screenshots here after deployment.

Dashboard **Network Explorer** **Recommendations**
(screenshot) *(screenshot)* *(screenshot)*

<div align="center">

Built with ❤️ using **CognoDB**, **Node.js**, and **React**

</div>